



Names, Bindings, and Scopes

CS3204N Programming Languages

Introduction

- Imperative programming languages are abstractions of the underlying von Neumann computer architecture.
- Memory cells are abstracted as **variables**.
- This chapter focuses on the fundamental attributes of variables and the concept of binding.

Names (Identifiers)

- **Names** are used to identify entities in programs.
- **Design issues:** Are names case sensitive? Are special words reserved words or keywords?
- **Length Limits:** Vary across languages (e.g., C99 has no limit, but only the first 63 characters are significant).
- **Keywords:** A word that is special only in certain contexts (e.g., Integer in Fortran).
- **Reserved words:** A special word that cannot be used as a user-defined name (improves readability).

Variables

- A **variable** is an abstraction of a computer memory cell or collection of cells.
- Variables are characterized by a sextuple of attributes: **Name, Address, Value, Type, Lifetime, and Scope.**
- Not all variables have names (e.g., explicit heap-dynamic variables).

Variable Attributes: Address

- **Address:** The memory address with which a variable is associated.
- **L-value:** The address of a variable, named because the address is required when a variable appears on the left side of an assignment.
- **Aliases:** Created when more than one variable name can be used to access the same memory location.
- Aliases hinder readability and are created via pointers, references, and unions.

Variable Attributes: Type & Value

- **Type:** Determines the range of values the variable can store and the set of operations defined for those values.
- **Value:** The contents of the memory cell or cells associated with the variable.
- **R-value:** The variable's value, named because it is required when a variable appears on the right side of an assignment.

The Concept of Binding

- **Binding:** An association between an attribute and an entity (e.g., between a variable and its type, or an operation and a symbol).
- **Binding Time:** The time at which a binding takes place.

The Concept of Binding

Possible Binding Times: * Language design time (e.g., binding * to multiplication).

- Language implementation time (e.g., binding int to a range of values).
- Compile time (e.g., binding a variable to a type in C).
- Load time (e.g., binding a static variable to a memory cell).
- Runtime (e.g., binding a local variable to a memory cell).

Static vs. Dynamic Binding

- **Static Binding:** A binding that first occurs before run time and remains unchanged throughout program execution.
- **Dynamic Binding:** A binding that first occurs during execution or can change during the course of execution

Type Binding Mechanisms

Static Type Binding:

- **Explicit declaration:** A statement that lists variable names and specifies their type.
- **Implicit declaration:** Associating variables with types through default conventions rather than declarations (e.g., Fortran starting letters).
- **Type inferencing:** The type is determined from the context of the initialization (e.g., `var sum = 0`; infers integer).

Dynamic Type Binding: The variable is bound to a type when it is assigned a value (e.g., Python, JavaScript).

Storage Bindings & Lifetime

- **Allocation:** Getting a cell from an available pool of memory cells.
- **Deallocation:** Putting a cell back into the pool.
- **Lifetime:** The time during which a variable is bound to a specific memory location.
- Begins with allocation and ends with deallocation.

Scalar Variables by Lifetime

- **Static variables:** Bound to memory cells before execution begins and remain bound to the same memory cell throughout execution. (Highly efficient, but no recursion support).
- **Stack-dynamic variables:** Storage bindings are created when their declaration statements are elaborated at runtime, but types are statically bound. (Supports recursion, but incurs runtime overhead for allocation).

Scalar Variables by Lifetime

- **Explicit heap-dynamic variables:** Allocated and deallocated by explicit directives specified by the programmer at runtime (e.g., new and delete operators in C++). Used for dynamic structures like trees and linked lists.
- **Implicit heap-dynamic variables:** Allocation and deallocation are caused implicitly by assignment statements (e.g., all variables in APL or strings in JavaScript). Highly flexible but inefficient.

Understanding Scope

- **Scope:** The range of statements in which a variable is **visible**.
- **Visible:** A variable is visible in a statement if it can be referenced in that statement.
- **Local variables:** Variables declared within a specific block or unit.
- **Nonlocal variables:** Variables visible in a unit but not declared there.
- **Global variables:** A special category of nonlocal variables accessible throughout the program.

Static Scoping

- **Static (Lexical) Scoping:** The scope of a variable can be statically determined prior to execution based on the spatial/textual layout of the code.
- **Static Ancestors:** Enclosing static scopes (subprograms nested inside others).
- **Blocks:** A method of creating localized static scopes inside program units (e.g., code within { ... }).

Dynamic Scoping

- **Dynamic Scoping:** Based on the calling sequence of subprograms, not on their textual relationship.
- Scope is determined at runtime.
- To resolve a nonlocal reference, the execution sequence is searched backward until a declaration is found.
- **Disadvantage:** Lowers readability and reliability; cannot statically type-check nonlocal references.

Scope vs. Lifetime

- Often confused, but distinctly different concepts.
- **Scope** is a spatial concept (a textual region of the program).
- **Lifetime** is a temporal concept (a period of time during program execution).
- *Example:* A C/C++ variable declared static inside a function has a lifetime of the entire program, but its scope is limited to that specific function.

Referencing Environments

- **Referencing Environment:** The collection of all names that are visible at a given statement in a program.
- In a **static-scoped** language: It consists of local variables plus all visible variables in all enclosing scopes.
- In a **dynamic-scoped** language: It consists of local variables plus all visible variables in all **active subprograms**.
- **Active subprogram:** A subprogram whose execution has begun but has not yet terminated.

Named Constants

- **Named Constant:** A variable that is bound to a value only when it is bound to storage (it cannot be changed later).
- **Manifest constants:** Named constants whose values are mapped at compile time.
- Used significantly to improve readability and to parameterize programs (e.g., using MAX_LENGTH instead of the literal 100).

Named Constants

- **Named Constant:** A variable that is bound to a value only when it is bound to storage (it cannot be changed later).
- **Manifest constants:** Named constants whose values are mapped at compile time.
- Used significantly to improve readability and to parameterize programs (e.g., using `MAX_LENGTH` instead of the literal 100).